



МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт
Информационных
технологий

Кафедра
Информационных
технологий
и вычислительных систем

ОТЧЕТ О ВЫПОЛНЕНИИ
ПРАКТИЧЕСКОЙ РАБОТЫ №2 ПО ДИСЦИПЛИНЕ
«Математическое и компьютерное моделирование»

СТУДЕНТА 1 КУРСА магистратуры ГРУППЫ ИДМ-26-08
(уровень профессионального образования)

ЗАМОСТЬЯНОВА НИКОЛАЯ ВЛАДИСЛАВОВИЧА

НА ТЕМУ
Метод наименьших квадратов, метод простых итераций

Направление: 09.04.04 Программная инженерия
Профиль Технологии разработки интеллектуальных систем и
подготовки: программных комплексов

Отчет сдан «__» _____ 2026г.

Оценка _____

Преподаватель Палванов Мурод Равшанович, ст. преподаватель
(Ф.И.О., должность, степень, звание.)

(подпись)

МОСКВА 2026

Скриншоты работы программы:

Метод наименьших квадратов и метод простых итераций
Оформление по решению семинара 2. Точность итераций $\varepsilon = 0,1$.

=====

Задание 1. Метод наименьших квадратов. Вариант 6

=====

Ищем тренд в виде $y = ax + b$.

1. Составляем расчётную таблицу

i	x _i	y _i	x _i ²	x _i y _i
1	4,92	-0,05	24,2064	-0,246
2	2,20	-7,76	4,84	-17,072
3	3,76	2,30	14,1376	8,648
4	2,89	-5,46	8,3521	-15,7794
5	3,21	-1,97	10,3041	-6,3237
6	2,61	-4,52	6,8121	-11,7972
7	1,74	-5,70	3,0276	-9,918
8	0,02	-6,15	0,0004	-0,123
9	0,45	-5,76	0,2025	-2,592
10	2,41	-3,27	5,8081	-7,8807
Σ	24,21	-38,34	77,6909	-63,084

Дополнительно: $\Sigma x = 24,21$, $n = 10$.

2. Находим коэффициент a

Для линейной аппроксимации

$$a = (n \cdot \Sigma x_i y_i - (\Sigma x_i)(\Sigma y_i)) / (n \cdot \Sigma x_i^2 - (\Sigma x_i)^2).$$

Подставляем значения:

$$\begin{aligned} a &= (10 \cdot (-63,084) - 24,21 \cdot (-38,34)) / (10 \cdot 77,6909 - (24,21)^2) \\ &= (-630,84 - (-928,2114)) / (776,909 - 586,1241) \\ &= 297,3714 / 190,7849 \\ &\approx 1,5587. \end{aligned}$$

3. Находим коэффициент b

$$b = (\Sigma y_i - a \cdot \Sigma x_i) / n.$$

$$\begin{aligned} b &= (-38,34 - 1,5587 \cdot 24,21) / 10 \\ &= (-38,34 - 37,736127) / 10 \\ &= -76,076127 / 10 \\ &\approx -7,6076. \end{aligned}$$

Если не округлять a на промежуточном шаге, получаем $b \approx -7,6075$.

Ответ: $y \approx 1,5587x - 7,6075$.

График сохранён: trend_var6.png

Рис. 1. Метод наименьших квадратов вариант 6

=====

Задание 1. Метод наименьших квадратов. Вариант 7

=====

Ищем тренд в виде $y = ax + b$.

1. Составляем расчётную таблицу

i	x _i	y _i	x _i ²	x _i y _i
1	3,50	-6,16	12,25	-21,56
2	1,93	-5,89	3,7249	-11,3677
3	2,11	-7,13	4,4521	-15,0443
4	6,34	-5,09	40,1956	-32,2706
5	0,54	-8,34	0,2916	-4,5036
6	7,83	-4,92	61,3089	-38,5236
7	0,31	-6,96	0,0961	-2,1576
8	4,44	-6,45	19,7136	-28,638
9	1,76	-7,24	3,0976	-12,7424
10	9,32	-2,82	86,8624	-26,2824
Σ	38,08	-61	231,9928	-193,0902

Дополнительно: $\Sigma x = 38,08$, $n = 10$.

2. Находим коэффициент a

Для линейной аппроксимации

$$a = (n \cdot \Sigma x_i y_i - (\Sigma x_i)(\Sigma y_i)) / (n \cdot \Sigma x_i^2 - (\Sigma x_i)^2).$$

Подставляем значения:

$$\begin{aligned} a &= (10 \cdot (-193,0902) - 38,08 \cdot (-61)) / (10 \cdot 231,9928 - (38,08)^2) \\ &= (-1930,902 - (-2322,88)) / (2319,928 - 1450,0864) \\ &= 391,978 / 869,8416 \\ &\approx 0,4506. \end{aligned}$$

3. Находим коэффициент b

$$b = (\Sigma y_i - a \cdot \Sigma x_i) / n.$$

$$\begin{aligned} b &= (-61 - 0,4506 \cdot 38,08) / 10 \\ &= (-61 - 17,158848) / 10 \\ &= -78,158848 / 10 \\ &\approx -7,8159. \end{aligned}$$

Если не округлять a на промежуточном шаге, получаем $b \approx -7,8160$.

Ответ: $y \approx 0,4506x - 7,8160$.

График сохранён: trend_var7.png

Рис. 2. Метод наименьших квадратов вариант 7

=====

Задание 1. Метод наименьших квадратов. Вариант 3

=====

Ищем тренд в виде $y = ax + b$.

1. Составляем расчётную таблицу

i	x _i	y _i	x _i ²	x _i y _i
1	3,27	-9,22	10,6929	-30,1494
2	9,46	-5,40	89,4916	-51,084
3	3,68	-8,84	13,5424	-32,5312
4	9,44	-4,68	89,1136	-44,1792
5	0,07	-10,58	0,0049	-0,7406
6	5,17	-6,70	26,7289	-34,639
7	2,73	-8,93	7,4529	-24,3789
8	0,24	-10,75	0,0576	-2,58
9	5,92	-6,96	35,0464	-41,2032
10	2,05	-9,17	4,2025	-18,7985
Σ	42,03	-81,23	276,3337	-280,284

Дополнительно: $\Sigma x = 42,03$, $n = 10$.

2. Находим коэффициент a

Для линейной аппроксимации

$$a = (n \cdot \Sigma x_i y_i - (\Sigma x_i)(\Sigma y_i)) / (n \cdot \Sigma x_i^2 - (\Sigma x_i)^2).$$

Подставляем значения:

$$\begin{aligned} a &= (10 \cdot (-280,284) - 42,03 \cdot (-81,23)) / (10 \cdot 276,3337 - (42,03)^2) \\ &= (-2802,84 - (-3414,0969)) / (2763,337 - 1766,5209) \\ &= 611,2569 / 996,8161 \\ &\approx 0,6132. \end{aligned}$$

3. Находим коэффициент b

$$b = (\Sigma y_i - a \cdot \Sigma x_i) / n.$$

$$\begin{aligned} b &= (-81,23 - 0,6132 \cdot 42,03) / 10 \\ &= (-81,23 - 25,772796) / 10 \\ &= -107,002796 / 10 \\ &\approx -10,7003. \end{aligned}$$

Если не округлять a на промежуточном шаге, получаем $b \approx -10,7003$.

Ответ: $y \approx 0,6132x - 10,7003$.

График сохранён: trend_var3.png

Рис. 3. Метод наименьших квадратов вариант 3

=====

Задание 2. Метод простых итераций. Первая система

=====

Решить с точностью $\varepsilon = 0,1$ систему

$$\begin{cases} x + y - 2z = 6, \\ 2x + 3y - 7z = 16, \\ 5x + 2y + z = 16. \end{cases}$$

1. Приводим систему к итерационному виду

Из первого уравнения:

$$x = -y + 2z + 6.$$

Из второго уравнения:

$$y = -(2/3)x + (7/3)z + 16/3.$$

Из третьего уравнения:

$$z = -5x - 2y + 16.$$

Итак,

$$x = -y + 2z + 6.$$

$$y = -(2/3)x + (7/3)z + 16/3.$$

$$z = -5x - 2y + 16.$$

2. Проверяем условие сходимости

Матрица итераций α :

$$\begin{matrix} 0 & -1 & 2 \\ -2/3 & 0 & 7/3 \\ -5 & -2 & 0 \end{matrix}$$

Свободный вектор β :

$$\begin{matrix} 6 \\ 16/3 \\ 16 \end{matrix}$$

Суммы модулей коэффициентов по строкам: 3, 3, 7.

Следовательно, $q = \|\alpha\| = 7 > 1$.

Достаточное условие сходимости не выполнено (нужно $q < 1$). Итерации не проводятся.

Система имеет единственное решение: $x = 3, y = 1, z = -1$.

Рис. 4. Метод простых итераций первая система

```

=====
Задание 2. Метод простых итераций. Вторая система
=====
Решить с точностью  $\varepsilon = 0,1$  систему

$$\begin{cases} 2x - y + 3z = -2, \\ 4x - 2y + 5z = 1, \\ 2x - y + z = 7. \end{cases}$$


1. Приводим систему к итерационному виду
Из первого уравнения:
 $x = 0,5y - 1,5z - 1.$ 
Из второго уравнения:
 $y = 2x + 2,5z - 0,5.$ 
Из третьего уравнения:
 $z = -2x + y + 7.$ 
Итак,
 $x = 0,5y - 1,5z - 1.$ 
 $y = 2x + 2,5z - 0,5.$ 
 $z = -2x + y + 7.$ 

2. Проверяем условие сходимости
Матрица итераций  $\alpha$ :

$$\begin{pmatrix} 0 & 0,5 & -1,5 \\ 2 & 0 & 2,5 \\ -2 & 1 & 0 \end{pmatrix}$$

Свободный вектор  $\beta$ :

$$\begin{pmatrix} -1 \\ -0,5 \\ 7 \end{pmatrix}$$

Суммы модулей коэффициентов по строкам: 2, 4,5, 3.

Следовательно,  $q = \|\alpha\| = 4,5 > 1.$ 
Достаточное условие сходимости не выполнено (нужно  $q < 1$ ). Итерации не проводятся.

Система несовместна.

```

Рис. 5. Метод простых итераций, вторая система

```

=====
Задание 2. Метод простых итераций. Третья система
=====
Решить с точностью  $\epsilon = 0,1$  систему

$$\begin{cases} x + 3y - 2z = 4, \\ 2x + 5y + z = 7, \\ x + y + 8z = 2. \end{cases}$$


1. Приводим систему к итерационному виду
Из первого уравнения:
 $x = -3y + 2z + 4.$ 
Из второго уравнения:
 $y = -0,4x - 0,2z + 1,4.$ 
Из третьего уравнения:
 $z = -0,125x - 0,125y + 0,25.$ 
Итак,
 $x = -3y + 2z + 4.$ 
 $y = -0,4x - 0,2z + 1,4.$ 
 $z = -0,125x - 0,125y + 0,25.$ 

2. Проверяем условие сходимости
Матрица итераций  $\alpha$ :


|        |        |      |
|--------|--------|------|
| 0      | -3     | 2    |
| -0,4   | 0      | -0,2 |
| -0,125 | -0,125 | 0    |


Свободный вектор  $\beta$ :


|      |
|------|
| 4    |
| 1,4  |
| 0,25 |


Суммы модулей коэффициентов по строкам: 5, 0,6, 0,25.

Следовательно,  $q = ||\alpha|| = 5 > 1.$ 
Достаточное условие сходимости не выполнено (нужно  $q < 1$ ). Итерации не проводятся.

Система имеет бесконечно много решений:  $x = 1 - 13t, y = 1 + 5t, z = t.$ 

```

Рис. 6. Метод простых итераций, третья система

Листинг программы:

```

import sys
from decimal import Decimal, ROUND_HALF_UP, getcontext
from fractions import Fraction
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager

getcontext().prec = 50
getcontext().rounding = ROUND_HALF_UP

```

```

try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

EPS = Fraction(1, 10)
MAX_ITER = 50
NAMES = ("x", "y", "z")

VARIANTS = [
    (
        6,
        ["4.92", "2.20", "3.76", "2.89", "3.21", "2.61", "1.74",
         "0.02", "0.45", "2.41"],
        ["-0.05", "-7.76", "2.30", "-5.46", "-1.97", "-4.52", "-5.70",
         "-6.15", "-5.76", "-3.27"],
    ),
    (
        7,
        ["3.50", "1.93", "2.11", "6.34", "0.54", "7.83", "0.31",
         "4.44", "1.76", "9.32"],
        ["-6.16", "-5.89", "-7.13", "-5.09", "-8.34", "-4.92", "-6.96",
         "-6.45", "-7.24", "-2.82"],
    ),
    (
        3,
        ["3.27", "9.46", "3.68", "9.44", "0.07", "5.17", "2.73",
         "0.24", "5.92", "2.05"],
        ["-9.22", "-5.40", "-8.84", "-4.68", "-10.58", "-6.70", "-8.93",
         "-10.75", "-6.96", "-9.17"],
    ),
]

SYSTEMS = [
    (
        "Первая система",
        [[1, 1, -2], [2, 3, -7], [5, 2, 1]],
        [6, 16, 16],
    ),
    (
        "Вторая система",
        [[2, -1, 3], [4, -2, 5], [2, -1, 1]],
        [-2, 1, 7],
    ),
    (
        "Третья система",
        [[1, 3, -2], [2, 5, 1], [1, 1, 8]],
        [4, 7, 2],
    ),
]

def fmt_dec(value, places=None):

```

```

        number = value if isinstance(value, Decimal) else
Decimal(value)
        if places is not None:
            number = number.quantize(Decimal(1).scaleb(-places))
            if number == 0:
                number = Decimal(0).quantize(Decimal(1).scaleb(-
places))
            text = format(number, "f")
            if places is None and "." in text:
                text = text.rstrip("0").rstrip(".")
            if text in ("-0", "-0.0"):
                text = "0"
            return text.replace(".", ",")

def fmt_real(value):
    number = Fraction(value)
    if number.denominator > 100000:
        return          fmt_dec(Decimal(number.numerator)          /
Decimal(number.denominator), 4)
    if number.denominator == 1:
        return str(number.numerator)
    reduced = number.denominator
    while reduced % 2 == 0:
        reduced //= 2
    while reduced % 5 == 0:
        reduced //= 5
    if reduced == 1:
        twos = fives = 0
        factor = number.denominator
        while factor % 2 == 0:
            factor //= 2
            twos += 1
        factor = number.denominator
        while factor % 5 == 0:
            factor //= 5
            fives += 1
        quotient =          Decimal(number.numerator)          /
Decimal(number.denominator)
        return f"{quotient:.{max(twos, fives)}f}".replace(".",
",")
    return f"{number.numerator}/{number.denominator}"

def join_terms(terms):
    chunks = []
    for coef, variable in terms:
        coef = Fraction(coef)
        if coef == 0:
            continue
        magnitude = fmt_real(abs(coef))
        if variable:
            if magnitude == "1":

```

```

        piece = variable
    elif "/" in magnitude:
        piece = f"({magnitude}){variable}"
    else:
        piece = f"{magnitude}{variable}"
else:
    piece = magnitude
if not chunks:
    chunks.append(piece if coef > 0 else f"-{piece}")
else:
    chunks.append((" + " if coef > 0 else " - ") + piece)
return " ".join(chunks) if chunks else "0"

def linear_in_t(const, coef):
    return join_terms([(const, ""), (coef, "t")])

def factor(value):
    text = fmt_dec(value)
    return f"({text})" if value < 0 else text

def subtraction(left, right):
    right_text = f"({fmt_dec(right)})" if right < 0 else
    fmt_dec(right)
    return f"{fmt_dec(left)} - {right_text}"

def equation_text(slope, intercept):
    left = f"{fmt_dec(slope, 4)}x"
    if intercept == 0:
        return f"y ≈ {left}"
    sign = "+" if intercept > 0 else "-"
    return f"y ≈ {left} {sign} {fmt_dec(abs(intercept), 4)}"

def print_table(headers, rows):
    widths = [max(len(cell) for cell in column) for column in
    zip(headers, *rows)]

    def render(cells):
        return " ".join(cell.rjust(width) for cell, width in
        zip(cells, widths))

    print(render(headers))
    print(render(["-" * width for width in widths]))
    for row in rows:
        print(render(row))

def choose_font():

```

```

        installed          =          {font.name          for          font          in
font_manager.fontManager.ttflist}
    for name in ("Segoe UI", "Arial", "DejaVu Sans"):
        if name in installed:
            return name
    return "DejaVu Sans"

def save_plot(variant, xs, ys, slope, intercept, equation):
    plt.rcParams["font.family"] = choose_font()
    plt.rcParams["axes.unicode_minus"] = False

    x_min, x_max = min(xs), max(xs)
    pad = (x_max - x_min) * 0.08 or 0.5
    grid = [x_min - pad + i * (x_max - x_min + 2 * pad) / 200 for
i in range(201)]
    line = [float(slope) * x + float(intercept) for x in grid]

    figure, axes = plt.subplots(figsize=(8, 5))
    axes.scatter(xs, ys, color="black", zorder=3, label="исходные
точки")
    axes.plot(grid, line, color="tab:blue", label="линия тренда")
    axes.set_xlabel("x")
    axes.set_ylabel("y")
    axes.set_title(f"Вариант {variant}: {equation}")
    axes.grid(True, linestyle="--", alpha=0.4)
    axes.legend()
    figure.tight_layout()

    path          =          Path(__file__).resolve().parent          /
f"trend_var{variant}.png"
    figure.savefig(path, dpi=150)
    plt.close(figure)
    print(f"График сохранён: {path.name}")

def least_squares(variant, x_text, y_text):
    xs = [Decimal(value) for value in x_text]
    ys = [Decimal(value) for value in y_text]
    n = len(xs)
    squares = [x * x for x in xs]
    products = [x * y for x, y in zip(xs, ys)]
    sum_x = sum(xs)
    sum_y = sum(ys)
    sum_x2 = sum(squares)
    sum_xy = sum(products)

    print("=" * 72)
    print(f"Задание 1. Метод наименьших квадратов. Вариант
{variant}")
    print("=" * 72)
    print("Ищем тренд в виде  $y = ax + b$ .")
    print()

```

```

print("1. Составляем расчётную таблицу")
headers = ["i", "x_i", "y_i", "x_i^2", "x_i y_i"]
rows = [
    [str(index),      fmt_dec(x,      2),      fmt_dec(y,      2),
fmt_dec(square), fmt_dec(product)]
    for index, (x, y, square, product) in enumerate(zip(xs,
ys, squares, products), start=1)
]
rows.append(["Σ",      fmt_dec(sum_x),      fmt_dec(sum_y),
fmt_dec(sum_x2), fmt_dec(sum_xy)])
print_table(headers, rows)
print(f"Дополнительно: Σx = {fmt_dec(sum_x)}, n = {n}.")
print()

print("2. Находим коэффициент a")
print("Для линейной аппроксимации")
print("a = (n·Σx_i y_i - (Σx_i)(Σy_i)) / (n·Σx_i^2 - (Σx_i)^2).")
print("Подставляем значения:")
product_xy = n * sum_xy
product_sums = sum_x * sum_y
product_x2 = n * sum_x2
square_sum = sum_x * sum_x
numerator = product_xy - product_sums
denominator = product_x2 - square_sum
slope = numerator / denominator
slope_rounded = slope.quantize(Decimal("0.0001"))
print(
    "a = "
    f"({factor(n)} · {factor(sum_xy)} -
{factor(sum_x)} · {factor(sum_y)}) / "
    f"({factor(n)} · {factor(sum_x2)} - ({fmt_dec(sum_x)})^2)"
)
print(f"      = {subtraction(product_xy, product_sums)} /
{subtraction(product_x2, square_sum)}")
print(f"      = {fmt_dec(numerator)} / {fmt_dec(denominator)}")
print(f"      ≈ {fmt_dec(slope_rounded, 4)}.")
print()

print("3. Находим коэффициент b")
print("b = (Σy_i - a·Σx_i) / n.")
product_rounded = slope_rounded * sum_x
difference_rounded = sum_y - product_rounded
intercept_rounded = difference_rounded / Decimal(n)
print(f"b = ({fmt_dec(sum_y)} - {fmt_dec(slope_rounded,
4)} · {factor(sum_x)}) / {n}")
print(f"      = {subtraction(sum_y, product_rounded)} / {n}")
print(f"      = {fmt_dec(difference_rounded)} / {n}")
print(f"      ≈ {fmt_dec(intercept_rounded, 4)}.")

intercept_exact = (sum_y - slope * sum_x) / Decimal(n)
intercept_shown = intercept_exact.quantize(Decimal("0.0001"))
print(

```

```

        "Если не округлять а на промежуточном шаге, "
        f"получаем  $b \approx \{fmt\_dec(intercept\_shown, 4)\}."$ 
    )
    print()
    equation = equation_text(slope_rounded, intercept_shown)
    print(f"Ответ: {equation}.")
    save_plot(variant, [float(x) for x in xs], [float(y) for y in
ys], slope_rounded, intercept_shown, equation)
    print()

def format_equation(coeffs, rhs):
    return join_terms(zip(coeffs, NAMES)) + " = " + fmt_real(rhs)

def print_original_system(matrix, rhs):
    lines = [format_equation(matrix[index], rhs[index]) for index
in range(3)]
    print(" { " + lines[0] + ",")
    print(" { " + lines[1] + ",")
    print(" { " + lines[2] + "."")

def iterative_form(matrix, rhs):
    alpha = []
    beta = []
    for index in range(3):
        pivot = Fraction(matrix[index][index])
        row = []
        for column in range(3):
            if column == index:
                row.append(Fraction(0))
            else:
                row.append(-Fraction(matrix[index][column]) /
pivot)
        alpha.append(row)
        beta.append(Fraction(rhs[index]) / pivot)
    return alpha, beta

def numeric_substitution(alpha_row, beta, previous):
    chunks = []
    for index, value in enumerate(previous):
        coef = alpha_row[index]
        if coef == 0:
            continue
        magnitude = fmt_real(abs(coef))
        piece = f"({fmt_real(value)})" if magnitude == "1" else
f"{magnitude} · ({fmt_real(value)})"
        if not chunks:
            chunks.append(piece if coef > 0 else f"-{piece}")
        else:
            chunks.append((" + " if coef > 0 else "- ") + piece)

```

```

if beta != 0 or not chunks:
    if not chunks:
        chunks.append(fmt_real(beta))
    elif beta > 0:
        chunks.append("+ " + fmt_real(beta))
    else:
        chunks.append("- " + fmt_real(abs(beta)))
return " ".join(chunks)

def rref(matrix, rhs):
    size = 3
    augmented = [
        [Fraction(matrix[row][column]) for column in range(size)]
+ [Fraction(rhs[row])]
        for row in range(size)
    ]
    pivots = []
    current = 0
    for column in range(size):
        selected = None
        for row in range(current, size):
            if augmented[row][column] != 0:
                selected = row
                break
        if selected is None:
            continue
        augmented[current], augmented[selected] =
augmented[selected], augmented[current]
        pivot = augmented[current][column]
        augmented[current] = [value / pivot for value in
augmented[current]]
        for row in range(size):
            if row == current or augmented[row][column] == 0:
                continue
            factor_value = augmented[row][column]
            augmented[row] = [
                augmented[row][index] - factor_value *
augmented[current][index]
                for index in range(size + 1)
            ]
        pivots.append(column)
        current += 1
    return augmented, pivots

def solution_remark(matrix, rhs):
    augmented, pivots = rref(matrix, rhs)
    for row in augmented:
        if all(value == 0 for value in row[:3]) and row[3] != 0:
            print("Система несовместна.")
            return
    if len(pivots) == 3:

```

```

        values = "", ".join(f"{NAMES[index]} = {fmt_real(augmented[index][3])}" for index in range(3))
        print(f"Система имеет единственное решение: {values}.")
        return

    column_to_row = {column: row for row, column in enumerate(pivots)}
    free = [index for index in range(3) if index not in column_to_row]
    if len(free) != 1:
        print("Система имеет бесконечно много решений.")
        return
    free_index = free[0]
    expressions = []
    for index in range(3):
        if index == free_index:
            expressions.append("t")
            continue
        row = augmented[column_to_row[index]]
        expressions.append(linear_in_t(row[3], -row[free_index]))
    described = "", ".join(f"{NAMES[index]} = {expressions[index]}"
for index in range(3))
    print(f"Система имеет бесконечно много решений: {described}.")

def simple_iterations(title, matrix, rhs):
    print("=" * 72)
    print(f"Задание 2. Метод простых итераций. {title}")
    print("=" * 72)
    print(f"Решить с точностью  $\varepsilon = \text{{fmt\_real(EPS)}}$  систему")
    print_original_system(matrix, rhs)
    print()

    alpha, beta = iterative_form(matrix, rhs)
    print("1. Приводим систему к итерационному виду")
    for index, name in enumerate(NAMES):
        print(f"Из {'первого', 'второго', 'третьего'}[index] уравнения:")
        print(f" {name} = {join_terms([*zip(alpha[index], NAMES), (beta[index], '')[index])}).")
        print("Итак,")
        for index, name in enumerate(NAMES):
            print(f" {name} = {join_terms([*zip(alpha[index], NAMES), (beta[index], '')[index])}).")
        print()

    print("2. Проверяем условие сходимости")
    print("Матрица итераций  $\alpha$ :")
    rendered = [[fmt_real(value) for value in row] for row in alpha]
    width = max(len(cell) for row in rendered for cell in row)
    for row in rendered:
        print(" " + " ".join(cell.rjust(width) for cell in row))

```

```

print("Свободный вектор  $\beta$ :")
for value in beta:
    print(f"    {fmt_real(value)}")
row_sums = [sum(abs(value) for value in row) for row in alpha]
sums_text = ", ".join(fmt_real(value) for value in row_sums)
print(f"Суммы модулей коэффициентов по строкам: {sums_text}.")
q = max(row_sums)
print()

if q < 1:
    print(f"Следовательно,  $q = ||\alpha|| = {fmt_real(q)} < 1.$ ")
    print("Метод сходится.")
    print()
    run_iterations(alpha, beta, q)
else:
    relation = "=" if q == 1 else ">"
    print(f"Следовательно,  $q = ||\alpha|| = {fmt_real(q)}$  {relation} 1.")
    print("Достаточное условие сходимости не выполнено (нужно  $q < 1$ ). Итерации не проводятся.")
    print()

solution_remark(matrix, rhs)
print()

def run_iterations(alpha, beta, q):
    threshold = EPS * (1 - q) / q
    current = list(beta)
    print("3. Выбираем начальное приближение")
    print("Берём  $X^{(0)} = \beta$ :")
    for name, value in zip(NAMES, current):
        print(f"    {name}^{(0)} = {fmt_real(value)}")
    print()
    print("Все новые значения вычисляются только по значениям предыдущей итерации.")

    for step in range(1, MAX_ITER + 1):
        updated = []
        for index in range(3):
            value = beta[index]
            for column in range(3):
                value += alpha[index][column] * current[column]
            updated.append(value)
        difference = sum(abs(updated[index] - current[index]) for index in range(3))
        print()
        print(f"Итерация {step}")
        for index, name in enumerate(NAMES):
            substitution = numeric_substitution(alpha[index], beta[index], current)
            print(f"    {name}^{({step})} = {substitution} = {fmt_real(updated[index])}")

```

```

        print(f"||X^{step} - X^{step - 1}||_1 = {fmt_real(difference)}.")
        print(f"ε(1 - q) / q = {fmt_real(threshold)}.")
        if difference <= threshold:
            relation = "≤"
            print(
                f"{fmt_real(difference)} {relation} {fmt_real(threshold)}, "
                "требуемая точность достигнута."
            )
            current = updated
            break
        print(f"{fmt_real(difference)} > {fmt_real(threshold)}, "
              "продолжаем.")
        current = updated
    else:
        print()
        print(f"За {MAX_ITER} итераций требуемая точность не достигнута.")
        return

    print()
    answer = ", ".join(
        f"{name} ≈ {fmt_dec(Decimal(value.numerator) / Decimal(value.denominator), 4)}"
        for name, value in zip(NAMES, current)
    )
    print(f"Ответ: {answer}.")

def main():
    print("Метод наименьших квадратов и метод простых итераций")
    print("Оформление по решению семинара 2. Точность итераций ε = 0,1.")
    print()
    for variant, xs, ys in VARIANTS:
        least_squares(variant, xs, ys)
    for title, matrix, rhs in SYSTEMS:
        simple_iterations(title, matrix, rhs)

if __name__ == "__main__":
    main()

```